

Informe Previo Desafío I

Sweet Crush

Contextualización, Análisis y Diseño

José Gabriel Giraldo Flórez

Informática II – Semestre 2026-2

Septiembre 2026

1 Contextualización

El Desafío I consiste en diseñar e implementar el juego *Sweet Crush* en C++ dentro del framework Qt, bajo restricciones estrictas de programación estructurada. Cada ficha del tablero debe almacenarse usando exactamente 3 bits, empaquetados de forma continua en un bloque de memoria dinámica (uso obligatorio), sin relleno para alinear cada ficha a un byte.

Restricciones y condicionamiento del diseño:

- Sin `struct`, `class`, `template`.
- Sin STL (nada de `vector`, `string`, `algorithm`, `map`, etc.).
- Sin objetos tipo *string* (incluye `QString`).
- Sin sintaxis ANSI C pura.
- Implementación obligatoria en Qt, con programación modular `.h/.cpp`.
- El tablero siempre se mantiene en bits; nunca se copia en un arreglo normal.
- Gestión “real” de memoria: crece o decrece según sus dimensiones, sin mantener un tamaño estático.

Estas restricciones corresponden de forma directa a los tres criterios de evaluación del desafío: 30% operadores de bits, 35% gestión de memoria, 35% lógica de juego.

2 Análisis del problema

2.1 Representación de fichas

Se implementan 6 tipos de fichas con 3 bits, generando 8 combinaciones posibles; quedan 2 libres para estados adicionales:

Código	Significado
000 – 101	FICHA_A .. FICHA_F
110	VACIO (posición libre, resultado de una eliminación)
111	ESPECIAL (a decisión propia; puede quedar sin usar)

2.2 Representación del tablero

El tablero se reserva como secuencia continua de bits, sin ningún tipo de relleno entre fichas:

```
pos = fila * columnas + columna    bit_inicial = pos * 3

byte = bit_inicial / 8    desfase = bit_inicial % 8

bits_necesarios = 3 * filas * columnas    bytes_necesarios = techo(bits_necesarios /
8)
```

Ya que cada ficha ocupa 3 bits y cada byte 8, las fronteras entre fichas no coinciden necesariamente con las fronteras entre bytes: una ficha puede quedar completamente contenida en un byte, o repartida entre 2 bytes consecutivos. El análisis debe tomar ambos casos de forma unificada.

Estrategia de lectura/escritura sin ramificar en casos: en lugar de tratar por separado fichas en un solo byte y fichas repartidas en dos bytes, se combina el byte actual y el siguiente en una **ventana de 16 bits**:

```
ventana = tablero[byte] | (tablero[byte + 1] << 8)
ficha = (ventana > desfase) & 0b111
```

Con el corrimiento y la máscara se extraen los 3 bits correctos independientemente del cruce de fronteras de bytes, sin necesidad de condicionales para los distintos escenarios. Es esencial no leer `byte + 1` cuando no exista (esto solo ocurre si la última ficha cae exactamente dentro de un único byte, con desfase ≤ 5). Para la escritura se usa el mismo principio: se limpia la ficha anterior con una máscara negada sobre la ventana de 16 bits, se inserta el nuevo valor con su corrimiento, y se separa el resultado en uno o dos bytes según el caso.

2.3 Gestión de memoria: regla del 65%

Se exige que la memoria reservada corresponda siempre a las dimensiones actuales del tablero, con una excepción explícita: al eliminar una fila o columna, la reasignación física de memoria solo debe ocurrir cuando la utilización caiga por debajo del 65% respecto a la capacidad actualmente reservada.

```
uso = bits_necesarios_nuevo / (bytes_asignados_actuales * 8)
```

```
si uso < 0.65: reasignar el buffer a bytes_necesarios(bits_necesarios_nuevo)
si no: conservar el buffer actual (queda con bytes sobrantes, sin usar, al final)
```

Los bits válidos siempre comienzan en el bit 0 del byte 0 (los índices siempre se calculan desde 0); los bits no utilizados quedan agrupados al final del arreglo, cumpliendo la exigencia de que las tramas válidas inicien en el bit menos significativo. Esto evita reasignaciones costosas cuando el usuario agrega y elimina filas o columnas repetidamente en tamaños cercanos.

2.4 Agregar/eliminar filas o columnas

Dado que las fichas no están alineadas por fila, agregar o quitar una fila o columna en la mitad del tablero desplaza casi todas las fichas posteriores en memoria. Por eso no se mueven los bits a mano caso por caso; en su lugar, se arma el tablero nuevo desde cero:

1. Se calculan las nuevas filas/columnas y la memoria necesaria (si se agrega, se reserva justo lo necesario; si se elimina, se aplica la regla del 65%).
2. Se reserva la nueva memoria.
3. Se recorre el tablero nuevo posición por posición: si la posición ya existía en el tablero antiguo, se copia la ficha; si es una posición nueva, se genera una ficha al azar.
4. Se libera el tablero viejo y se actualizan filas, columnas y tamaño en memoria.

Así se sigue usando las funciones de extraer e insertar bits en cada paso, sin programar a mano los casos en que una ficha queda partida entre dos bytes.

2.5 Detección de combinaciones y cascadas

Una combinación requiere una corrida de 3 o más fichas iguales, consecutivas en sentido horizontal o vertical. Una ficha que participe de una combinación horizontal y vertical simultáneamente debe procesarse una única vez, lo cual requiere **marcar antes de eliminar**, en lugar de eliminar directamente durante la detección.

Una **cascada** se da cuando, tras una eliminación y su reorganización consecuente (gravedad y relleno de celdas), se generan nuevas combinaciones sin intervención del jugador. Este proceso se repite hasta que no se generen más combinaciones de forma automática.

2.6 Problemas de desarrollo previstos

- Verificación de extracción/inserción de bits en los índices que crucen fronteras entre bytes.
- Corrección del cálculo de la regla del 65% en eliminaciones sucesivas de filas o columnas.
- Verificación de que una ficha en cruce horizontal-vertical se elimine una única vez y se cuente correctamente en las estadísticas.
- Consistencia al insertar o eliminar filas/columnas en posiciones intermedias (no solo en los extremos).

3 Diseño de la solución

3.1 Arquitectura de módulos

El proyecto se organiza en módulos `.h/.cpp` independientes, cada uno correspondiente a una función explícita del enunciado o de los criterios de evaluación.

Módulo	Responsabilidad
<code>bits.h / bits.cpp</code>	Primitivas genéricas de bits (base de todo el proyecto)
<code>tablero.h / tablero.cpp</code>	Creación/liberación del tablero, obtener/establecer fichas
<code>fichas.h / fichas.cpp</code>	Códigos, generación aleatoria, símbolos visuales
<code>combinaciones.h / .cpp</code>	Detección horizontal/vertical, marcado, eliminación
<code>reorganizacion.h / .cpp</code>	Gravedad, relleno, bucle de cascadas
<code>estructura.h / .cpp</code>	Agregar/eliminar filas o columnas, regla del 65%
<code>estado.h / estado.cpp</code>	Contador de partida y puntuación
<code>visualizacion.h / .cpp</code>	Mostrar tablero (fichas y binario), estadísticas

3.2 Funciones más importantes por módulo

bits.h / bits.cpp – funciones base que permiten acceder al arreglo de bytes del tablero; todos los módulos pasan por ellas:

- `extraerBits3(buffer, bitInicial) / insertarBits3(buffer, bitInicial, valor)`
- `bitsNecesarios(filas, columnas) / bytesNecesarios(bits)`

tablero.h / tablero.cpp: `crearTablero(...)` / `liberarTablero(...)` / `obtenerFicha(...)` / `establecerFicha(...)`

combinaciones.h / combinaciones.cpp – el arreglo `marcador` es una estructura auxiliar temporal (no una copia de las fichas); ambas detecciones hacen OR sobre el mismo arreglo para que una ficha en cruce H-V quede marcada una sola vez: `detectarHorizontales(...)` / `detectarVerticales(...)` / `eliminarMarcadas(...)`

reorganizacion.h / reorganizacion.cpp: `aplicarGravedad(...)` / `rellenarVacios(...)` / `procesarCascadas(...)`

estructura.h / estructura.cpp: `agregarFila` / `eliminarFila` / `agregarColumna` / `eliminarColumna` / `decidirRedimensionar(...)` (aplica la regla del 65%)

estado.h / estado.cpp – módulo con variables internas y funciones de acceso: `registrarEliminacionManual`, `registrarFichasEliminadas`, `registrarCombinacion`, `registrarCascadas`, `sumarPuntuacion`, y sus respectivos `obtener...`

visualizacion.h / visualizacion.cpp: `mostrarTableroFichas`, `mostrarTableroBinario`, `mostrarEstadisticas`.

3.3 Criterio de puntuación

Eliminación manual sin combo resultante: 0 puntos. Cada ficha eliminada dentro de una combinación: 10 puntos, multiplicados por el nivel de cascada (cascada 1 $\rightarrow \times 1$, cascada 2 $\rightarrow \times 2$, cascada 3 $\rightarrow \times 3$, etc.), premiando las reacciones en cadena.

Nota importante: este es un diseño base con las principales implementaciones propuestas, teniendo en cuenta las restricciones y exigencias del trabajo. Es posible incorporar funciones adicionales no documentadas aquí durante el avance del desarrollo.